Package 'cloudscape'

August 5, 2026

Type Package

Title Clear-Observation Availability and Cloud Analysis for Optical Satellite Image Time Series

Version 0.1.0

Description A multi-sensor framework that turns cloud statistics into decision-relevant measures of whether an optical time-series study is feasible at a given location. Statistics are aggregated onto an equal-area analysis grid rather than onto sensor footprints, removing the latitude bias that affects per-scene counts and making 'Landsat' and 'Sentinel-2' directly comparable. Provides interchangeable cloud-probability algorithms including a threshold baseline and a decision-cascade formulation following Zhu and Woodcock (2012) <doi:10.1016/j.rse.2011.10.028>, a geometric cloud-shadow projection engine with explicit separation of terrain shadow, synthetic cloud simulation for algorithm benchmarking, and propagation of observation loss into phenological retrieval error in days. Data access uses SpatioTemporal Asset Catalogs. Every statistic carries its unit, sample size and denominator, which are validated on construction.

License GPL-3

Encoding UTF-8

Depends R (>= 4.1)

Imports stats,
utils,
tools,
methods

Suggests terra (>= 1.7.0),
sf (>= 1.0.0),
httr2,
jsonlite,
data.table,
arrow,
ggplot2,
ranger,
xgboost,
torch,

future.apply,
mapgl,
shiny,
knitr,
rmarkdown,
testthat (>= 3.0.0),
httptest2,
withr

VignetteBuilder knitr

Config/testthat/edition 3

URL https://github.com/ehsanrahimi666/cloudscape

BugReports https://github.com/ehsanrahimi666/cloudscape/issues

RoxygenNote 7.3.2

Roxygen list(markdown = TRUE)

R topics documented:

`cloudscape-package` *cloudscape: Clear-Observation Availability and Cloud Analysis for Optical Satellite Image Time Series*

Description

A multi-sensor framework that turns cloud statistics into decision-relevant measures of whether an optical time-series study is feasible at a given location. Statistics are aggregated onto an equal-area analysis grid rather than onto sensor footprints, removing the latitude bias that affects per-scene counts and making Landsat and Sentinel-2 directly comparable.

Where to start

- `cl_grid` defines the equal-area analysis grid.
- `cl_search` queries a catalogue; `cl_items_to_obs` maps results onto the grid.
- `cl_clear_obs`, `cl_gaps` and `cl_seasonality` summarise availability.
- `cl_persistence` measures cloud autocorrelation.
- `cl_pheno_power` converts an acquisition pattern into phenological retrieval error in days.
- `cl_probability` and `cl_shadow` detect cloud and cloud shadow.
- `cl_simulate` and `cl_compare` benchmark detectors against known truth.

Author(s)

Ehsan Rahimi <ehsanrahimi666@gmail.com> and Chuleui Jung

`cl_agreement` *Agreement among several masks*

Description

Agreement among several masks

Usage

```
cl_agreement(masks)
```

Arguments

masks	Named list of cl_maskset objects or 0/1 matrices.

Value

A list with agreement (fraction of methods calling cloud), consensus (majority mask), entropy (binary entropy of the vote) and unanimous (logical matrix).

Examples

```
a <- matrix(c(1,1,0,0), 2, 2); b <- matrix(c(1,0,0,0), 2, 2)
cl_agreement(list(a = a, b = b))$agreement
```

---

cl_app *Launch the interactive scene explorer*

Description

Live imagery browsing needs a server: asset URLs from most catalogues are signed and expire, and masks are computed on demand. This is therefore a Shiny application rather than a static file.

Usage

```
cl_app(stats = NULL, ...)
```

Arguments

stats	Optional cl_stats to seed the map.
...	Passed to shiny::runApp().

Value

Invisibly NULL.

cl_apply_chunks *Apply a function over chunks, optionally in parallel*

Description

Apply a function over chunks, optionally in parallel

Usage

```
cl_apply_chunks(x, chunks, fun, workers = NULL)
```

Arguments

x	Object passed to `fun` along with each chunk.
chunks	A chunk plan from `cl_chunk`.
fun	Function with signature `(x, row_start, n_rows)`.
workers	Number of workers; 1 runs sequentially.

Value

A list of results, one per chunk.

cl_band *Extract a band by standardised name*

Description

Extract a band by standardised name

Usage

```
cl_band(x, band, required = TRUE)
```

Arguments

x	A `cl_scene`.
band	Standardised band name, e.g. `"nir"`.
required	Error if the band is absent.

Value

A `SpatRaster` layer, or `NULL`.

cl_bands *Band names present in a scene*

Description

Band names present in a scene

Usage

```
cl_bands(x)
```

Arguments

x	A cl_scene.

Value

Character vector of standardised band names.

cl_benchmark *Load a benchmark dataset*

Description

Benchmark imagery is too large to ship on CRAN and is therefore distributed through the companion data package. This function resolves the request, checks the cache, and gives an actionable message rather than failing obscurely when the data are absent.

Usage

```
cl_benchmark(dataset = c("synthetic", "cloudsen12", "kappaset", "biome"), n
  = 20L, cache = cl_cache_dir())
```

Arguments

dataset	One of "cloudsen12", "kappaset", "biome", "synthetic".
n	Number of patches to return.
cache	Cache directory.

Value

For "synthetic", a list of simulated scenes generated on the fly. For the others, the cached dataset if present, otherwise an informative error.

cl_cache_dir *Locate the cloudscape cache directory*

Description

Downloaded assets, resumable harvest state and companion datasets are stored here. The location follows, in order of precedence: the `cache_dir` option, the `CLOUDSCAPE_CACHE` environment variable, then a user-level cache path.

Usage

```
cl_cache_dir(create = TRUE)
```

Arguments

`create`	Create the directory if it does not exist.

Value

Absolute path to the cache directory.

cl_calibration *Probability calibration*

Description

A detector can discriminate well and still be badly calibrated, in which case its probabilities cannot be used to weight observations or propagate uncertainty. This reports the reliability curve together with the Brier score and expected calibration error.

Usage

```
cl_calibration(prob, truth, bins = 10L)
```

Arguments

`prob`	Predicted probabilities.
`truth`	Binary reference.
`bins`	Number of probability bins.

Value

A data frame with one row per bin, with `brier` and `ece` attributes.

cl_catalog *Configure or inspect a catalogue backend*

Description

Configure or inspect a catalogue backend

Usage

```
cl_catalog(backend = NULL)
```

Arguments

backend	One of "element84", "planetary", "cdse".

Value

A list describing the backend.

Examples

```
cl_catalog("element84")$url
```

cl_chunk *Plan chunked processing of a large raster*

Description

Returns row blocks sized to the configured pixel budget. Working in blocks is what allows continental analysis on ordinary hardware; the block size is exposed rather than hidden so that it can be tuned to available memory.

Usage

```
cl_chunk(nrow, ncol, n_layers = 1L, budget = NULL)
```

Arguments

nrow	Raster dimensions.
ncol	Raster dimensions.
n_layers	Number of layers held simultaneously.
budget	Target pixels in memory at once.

Value

A data frame with block, row_start, n_rows.

Examples

```
cl_chunk(10000, 10000, n_layers = 8)
```

`cl_classes` *Standard class coding*

Description

cloudscape uses one class vocabulary across all sensors and algorithms so that masks from Fmask, Sen2Cor SCL, s2cloudless and neural models can be compared directly.

Usage

```
cl_classes()
```

Value

A named integer vector.

Examples

```
cl_classes()
```

`cl_clear_obs` *Clear observation counts per cell and period*

Description

Clear observation counts per cell and period

Usage

```
cl_clear_obs(obs, by = c("year", "month", "season", "all"), threshold = 0.2,
  model = c("independent", "clustered", "empirical"), grid = NULL)
```

Arguments

`obs`	A `cl_obs` table.
`by`	Period granularity: `"year"`, `"month"`, `"season"` or `"all"`.
`threshold`	Maximum cloud fraction for an observation to count as usable.
`model`	Clear-probability model, see `cl_clear_prob`.
`grid`	Optional `cl_grid` recorded on the output.

Value

A `cl_stats` table with metrics `n_scenes`, `n_clear_obs`, `cloud_fraction` and `p_clear`.

Examples

```
set.seed(1)
o <- cl_obs(cell = rep(1:2, each = 40),
            date = rep(seq(as.Date("2023-01-01"), by = "9 days", length.out = 40), 2),
            cloud_fraction = c(runif(40, 0, .4), runif(40, .5, 1)),
            sensor = "sentinel-2-msi")
summary(cl_clear_obs(o, by = "year"))
```

`cl_clear_prob` *Probability that a pixel in a cell is clear*

Description

Converting a scene-level cloud fraction into a per-pixel clear probability requires an assumption, and the choice of assumption matters more than most users expect.

`"independent"` sets `P(clear) = 1 - cloud_fraction`. This treats cloud as spatially random within the footprint. Cloud is strongly clustered, so this systematically **overestimates** the number of usable observations for a specific location: a scene reported as 50 percent cloudy is far more often half covered by one cloud deck than uniformly half-obscured. It is the assumption implicit in any analysis that uses scene cloud percentage as a usability measure, and it is why tier A results should be read as an upper bound.

`"clustered"` applies a Beta-distributed correction with concentration `kappa`, reproducing the observed U-shaped distribution of pixel-level cloud occurrence within partly cloudy scenes.

`"empirical"` requires tier `"qa"` or `"mask"` data, where the fraction was measured for the cell rather than inferred from the scene, and applies no correction.

Usage

```
cl_clear_prob(obs, model = c("independent", "clustered", "empirical"), kappa
  = 0.5)
```

Arguments

`obs`	A `cl_obs` table.
`model`	One of `"independent"`, `"clustered"`, `"empirical"`.
`kappa`	Beta concentration for `"clustered"`; lower means more clustered. Values near 0.5 match reported pixel-scale behaviour.

Value

Numeric vector of clear probabilities, one per row of `obs`.

cl_cloud_height *Estimate cloud-top height from thermal brightness temperature*

Description

Uses the classical moist-adiabatic approach: the temperature difference between the cloud and the surrounding clear sky, divided by an assumed lapse rate, gives a height. The result is bracketed by physically plausible limits because the lapse rate is an assumption, not a measurement, and because thermal contrast collapses over cold surfaces.

Sensors without a thermal band return the supplied `default_range` instead, which is the honest answer: for Sentinel-2 the cloud-top height genuinely cannot be retrieved from the imagery alone and must be searched over.

Usage

```
cl_cloud_height(bt = NULL, cloud, clear = NULL, lapse_rate = 0.0065, limits
  = c(200, 12000), default_range = c(200, 12000))
```

Arguments

`bt`	Brightness temperature matrix in Kelvin, or `NULL`.
`cloud`	Logical cloud matrix.
`clear`	Optional logical matrix of confidently clear pixels; defaults to the complement of `cloud`.
`lapse_rate`	Environmental lapse rate in K/m.
`limits`	Physically plausible height range in metres.
`default_range`	Height range returned when no thermal band exists.

Value

A list with `height` (best estimate, `NA` if unavailable) and `range` (a length-2 search interval in metres).

cl_compare *Compare methods with uncertainty*

Description

Paired comparison of two or more masks against a common reference. Reports bootstrap confidence intervals on overall accuracy for each method, and McNemar's test for each pair, so that a difference is only claimed when the paired evidence supports it.

Usage

```
cl_compare(preds, truth, n_boot = 500L, conf = 0.95, block = NULL)
```

Arguments

preds	Named list of predicted masks.
truth	Reference mask.
n_boot	Bootstrap replicates.
conf	Confidence level.
block	Optional integer vector of block ids for a block bootstrap. Cloud masks are spatially autocorrelated, so a pixel-wise bootstrap understates uncertainty; supplying tile ids gives an honest interval.

Value

A list with `summary` (per-method accuracy and interval) and `pairwise` (McNemar statistics and p-values).

Examples

```
set.seed(3)
truth <- matrix(rbinom(1000, 1, .35), 25, 40)
a <- truth; a[sample(1000, 60)] <- 1 - a[sample(1000, 60)]
b <- truth; b[sample(1000, 120)] <- 1 - b[sample(1000, 120)]
cl_compare(list(good = a, worse = b), truth, n_boot = 200)$summary
```

`cl_confusion` *Confusion matrix*

Description

Confusion matrix

Usage

```
cl_confusion(pred, truth, levels = NULL, labels = NULL)
```

Arguments

pred	Predicted class codes or logical mask.
truth	Reference class codes or logical mask.
levels	Optional class levels; defaults to the union observed.
labels	Optional class labels.

Value

A table with predictions in rows and reference in columns.

cl_cube *Construct a lazy scene collection*

Description

A `cl_cube` never holds pixels. It holds an index of scenes plus a template describing the target geometry, and materialises blocks on demand. This is what allows a decade of imagery to be summarised without exhausting memory.

Usage

```
cl_cube(scenes, template = NULL, grid = NULL, manifest = NULL)
```

Arguments

`scenes`	A list of `cl_scene` objects, or a data frame index with at least the columns `id`, `datetime`, `sensor` and `href`.
`template`	Optional `SpatRaster` defining target grid, extent and CRS.
`grid`	Optional `cl_grid` for statistical aggregation.
`manifest`	A `cl_manifest`.

Value

An object of class `cl_cube`.

cl_disagreement *Pairwise disagreement between masks*

Description

Pairwise disagreement between masks

Usage

```
cl_disagreement(masks, strata = NULL)
```

Arguments

`masks`	Named list of masks.
`strata`	Optional factor or matrix of stratum labels, for example a land-cover or snow layer, used to report where methods diverge.

Value

A list with `pairwise` (a data frame of agreement, Jaccard and commission/omission relative to each other) and, when `strata` is given, `by_stratum`.

cl_explore *Export an interactive map of grid statistics*

Description

Writes a self-contained HTML file showing one metric per grid cell. The output has no server dependency and no expiring URLs, so it can be archived alongside a manuscript and will still work years later. Live imagery browsing is a different problem with different constraints and belongs in `cl_app`.

Usage

```
cl_explore(stats, metric, period = NULL, file = "cloudscape.html", grid =
  NULL, palette = c("#2c7bb6", "#ffffbf", "#d7191c"), title = NULL)
```

Arguments

`stats`	A `cl_stats` table.
`metric`	Metric to display.
`period`	Optional period filter.
`file`	Output path.
`grid`	The `cl_grid` the cells refer to; taken from `stats` if absent.
`palette`	Colour ramp endpoints.
`title`	Map title.

Value

The output path, invisibly.

cl_export *Export cloudscape objects*

Description

Export cloudscape objects

Usage

```
cl_export(x, file, format = c("auto", "csv", "rds", "geojson", "tif"))
```

Arguments

`x`	A `cl_stats`, `cl_maskset` or `cl_obs`.
`file`	Output path.
`format`	Output format; `"auto"` infers from the extension.

Value

The output path, invisibly.

`cl_feasibility` *Sampling design feasibility*

Description

Given a clear-sky probability and an acquisition cadence, reports the probability of obtaining at least `n_required` usable observations within a window, and the probability that no gap exceeds `max_gap`.

Usage

```
cl_feasibility(p_clear, n_acquisitions, n_required, window_days = 365,
  max_gap = NULL, n_sim = 2000)
```

Arguments

`p_clear`	Per-acquisition probability that the cell is usable.
`n_acquisitions`	Number of acquisition opportunities in the window.
`n_required`	Minimum usable observations needed.
`window_days`	Length of the window in days.
`max_gap`	Maximum tolerable gap in days, or `NULL` to skip.
`n_sim`	Monte Carlo replicates for the gap calculation.

Value

A list with `p_sufficient`, `expected_obs`, and, when `max_gap` is given, `p_gap_ok`.

Examples

```
cl_feasibility(p_clear = 0.35, n_acquisitions = 36, n_required = 12,
               window_days = 365, max_gap = 45)
```

cl_gaps *Cloud-gap statistics*

Description

Mean cloud cover is a poor summary of whether a time series is workable. What breaks a phenology retrieval is not the average, it is the longest stretch with no usable observation. `cl_gaps()` reports that stretch, its distribution, and the return period of gaps exceeding a stated length.

Gaps at the boundaries of the analysis window are included by default, because a cell whose first clear observation arrives in April has genuinely lost the spring green-up regardless of how many observations follow.

Usage

```
cl_gaps(obs, threshold = 0.2, by = c("year", "all", "season"), include_edges
  = TRUE, critical = 30)
```

Arguments

`obs`	A `cl_obs` table.
`threshold`	Maximum cloud fraction for a usable observation.
`by`	Period granularity for reporting.
`include_edges`	Count the intervals from period start to first clear observation and from last clear observation to period end.
`critical`	Gap length in days whose exceedance frequency is reported.

Value

A `cl_stats` table with `max_gap_days` and `mean_gap_days`, carrying an attribute `exceedance` giving the fraction of periods per cell in which `critical` was exceeded.

cl_geometry *Attach or compute scene geometry*

Description

Attach or compute scene geometry

Usage

```
cl_geometry(x, lon = NULL, lat = NULL)
```

Arguments

x	A cl_scene.
lon	Scene centre, used when metadata angles are absent.
lat	Scene centre, used when metadata angles are absent.

Value

The scene with solar_geom populated.

---

cl_grid *Define an equal-area analysis grid*

Description

Define an equal-area analysis grid

Usage

```
cl_grid(res = NULL, crs = NULL, extent = "global")
```

Arguments

res	Cell size in metres. Common choices are 100000 for global overviews, 25000 for continental analysis and 5000 for regional work.
crs	EPSG code of the equal-area CRS. Only 6933 is supported by the built-in closed-form projection; other codes require sf.
extent	Either "global" or a numeric vector c(xmin, ymin, xmax, ymax) in **degrees**.

Value

An object of class cl_grid.

Examples

```
g <- cl_grid(res = 100000)
g
nrow(cl_grid_cells(g)) > 0
```

cl_grid_cells *Cell identifiers, centroids and areas*

Description

Cells are numbered row-major from the top-left, matching raster convention. Because the grid is equal-area, every cell has an area of exactly `res^2` square metres regardless of latitude; this is the property that removes the latitude bias present in per-scene statistics.

Usage

```
cl_grid_cells(grid, cells = NULL)
```

Arguments

`grid`	A `cl_grid`.
`cells`	Optional integer vector of cell ids; defaults to all cells. Supplying ids avoids materialising a global table.

Value

A data frame with `cell`, `row`, `col`, `x`, `y`, `lon`, `lat`, `area_km2`.

cl_grid_index *Index sensor footprints onto the analysis grid*

Description

Converts a scene or tile footprint into a set of grid cells with weights. Two methods are available. `"centroid"` assigns a cell to a footprint when the cell centroid falls inside it; it is unbiased in expectation, needs no geospatial dependencies, and is accurate when cells are much smaller than footprints (the usual case: 25 km cells against 180 km Landsat scenes). `"area"` computes exact fractional overlap and requires `sf`; use it when cell size approaches footprint size.

Usage

```
cl_grid_index(footprint, grid, method = c("centroid", "area"))
```

Arguments

`footprint`	Either a two-column matrix or data frame of polygon vertex longitudes and latitudes, or an `sf` object.
`grid`	A `cl_grid`.
`method`	`"centroid"` or `"area"`.

Value

A data frame with columns `cell` and `weight`, where `weight` is the fraction of the cell covered by the footprint (always 1 for `method = "centroid"`).

Examples

```
g <- cl_grid(res = 100000)
poly <- cbind(lon = c(0, 2, 2, 0, 0), lat = c(50, 50, 52, 52, 50))
head(cl_grid_index(poly, g))
```

`cl_grid_lookup` *Locate cells from coordinates*

Description

Locate cells from coordinates

Usage

```
cl_grid_lookup(grid, lon, lat)
```

Arguments

`grid`	A `cl_grid`.
`lon`	Coordinates in degrees.
`lat`	Coordinates in degrees.

Value

Integer vector of cell ids, `NA` outside the grid.

`cl_indices` *Common spectral indices*

Description

Common spectral indices

Usage

```
cl_indices(bands, which = c("ndvi", "ndsi", "ndwi", "whiteness", "hot"))
```

Arguments

`bands`	Named list of reflectance matrices.
`which`	Indices to compute.

Value

A named list of matrices.

Examples

```
b <- list(green = matrix(.1,4,4), red = matrix(.08,4,4),
          nir = matrix(.35,4,4), swir16 = matrix(.2,4,4), blue = matrix(.09,4,4))
names(cl_indices(b))
```

---

cl_items_to_obs *Convert catalogue items to an observation table*

Description

Maps scene footprints onto the equal-area analysis grid and attaches the scene-level cloud fraction to every intersecting cell. This is tier A: fast and global, but the cloud fraction is a scene average, so the resulting clear-observation counts should be read as an optimistic bound. See `cl_clear_prob`.

Usage

```
cl_items_to_obs(items, grid, method = "centroid")
```

Arguments

`items`	A `cl_items` table.
`grid`	A `cl_grid`.
`method`	Footprint indexing method, see `cl_grid_index`.

Value

A `cl_obs` table with tier `"metadata"`. Items whose cloud-cover property is missing are excluded with a warning rather than silently treated as clear.

---

cl_manifest *Create or extract a provenance manifest*

Description

Satellite archives are reprocessed. Collection 2 changed Landsat cloud masking relative to Collection 1, and successive Sen2Cor baselines changed the Sentinel-2 scene classification layer. A statistic computed today may therefore differ from the same statistic computed on the same nominal data next year. Every cloudscape object carries a manifest recording exactly what was queried, when, and with which algorithm, so results remain auditable.

Usage

```
cl_manifest(x = NULL, ...)
```

Arguments

x	An object with a manifest, or NULL to build a fresh one.
...	Named fields to record.

Value

An object of class cl_manifest: a named list.

Examples

```
m <- cl_manifest(NULL, source = "element84", collection = "sentinel-2-l2a")
m$collection
```

---

cl_maskset *Construct a cloud/shadow mask set*

Description

Keeping probability and class in one object with the threshold and method recorded means a mask can always be re-thresholded without recomputation, and that comparisons between methods carry their own provenance.

Usage

```
cl_maskset(probability, class = NULL, shadow = NULL, method = NA_character_,
  threshold = NA_real_, manifest = NULL, ...)
```

Arguments

probability	SpatRaster or numeric matrix of cloud probability in \[0, 1\].
class	Optional integer SpatRaster/matrix of class codes. See cl_classes for the coding.
shadow	Optional shadow probability layer.
method	Name of the generating algorithm.
threshold	Probability threshold used to derive class.
manifest	A cl_manifest.
...	Additional algorithm parameters, recorded verbatim.

Value

An object of class cl_maskset.

cl_methods *List available cloud-probability methods*

Description

List available cloud-probability methods

Usage

```
cl_methods(sensor = NULL)
```

Arguments

sensor	Optional sensor id; when supplied, only methods whose required bands the sensor provides are returned.

Value

A data frame of methods.

Examples

```
cl_methods()
cl_methods(sensor = "sentinel-2-msi")   # no thermal-based methods
```

cl_method_register *Register a cloud-probability method*

Description

Register a cloud-probability method

Usage

```
cl_method_register(name, fun, requires, optional = character(), type =
  "physical", reference = NA_character_, overwrite = FALSE)
```

Arguments

name	Method name.
fun	Function with signature `(bands, geom, drv, ...)` returning a numeric matrix of probabilities in \[0, 1\].
requires	Standardised band names the method needs.
optional	Bands the method uses if present.
type	`"physical"`, `"statistical"` or `"learned"`.
reference	Short citation.
overwrite	Replace an existing registration.

Value

The method name, invisibly.

`cl_metrics` *The controlled metric vocabulary*

Description

The controlled metric vocabulary

Usage

```
cl_metrics()
```

Value

A data frame of metric names, units and permitted ranges.

`cl_obs` *Build an observation table*

Description

Build an observation table

Usage

```
cl_obs(cell, date, cloud_fraction, sensor = NA_character_, platform =
  NA_character_, tier = "metadata", shadow_fraction = NA_real_,
  snow_fraction = NA_real_)
```

Arguments

`cell`	Integer grid cell ids.
`date`	Acquisition dates.
`cloud_fraction`	Cloud fraction in \[0, 1\] for the cell.
`sensor`	Sensor ids.
`platform`	Optional platform names.
`tier`	Which evidence tier the cloud fractions came from: `"metadata"` (scene-level catalogue property), `"qa"` (aggregated quality layer) or `"mask"` (computed mask).
`shadow_fraction`	Optional additional fractions.
`snow_fraction`	Optional additional fractions.

Value

An object of class `cl_obs`, inheriting from `data.frame`.

`cl_options` *Get or set package options*

Description

`cl_options()` is the single configuration entry point for cloudscape. Called with no arguments it returns the full option list. Called with named arguments it sets them and returns the previous values invisibly, so it can be used with [base::on.exit()] to make temporary changes.

Usage

```
cl_options(...)
```

Arguments

`...`	Named options to set. Supported names: \describe{ \item{`catalog`}{Default STAC backend, one of `"element84"`, `"planetary"`, `"cdse"`.} \item{`grid_res`}{Default analysis grid resolution in metres.} \item{`grid_crs`}{EPSG code of the equal-area analysis CRS. Defaults to 6933 (EASE-Grid 2.0 global).} \item{`chunk_pixels`}{Target pixels per processing block.} \item{`workers`}{Number of parallel workers.} \item{`cache_dir`}{Directory for downloaded assets and harvest state.} \item{`timeout`}{HTTP timeout in seconds.} \item{`max_page`}{Maximum STAC items requested per page.} \item{`verbose`}{Emit progress messages.} }

Value

A named list of options. When setting, the previous values are returned invisibly.

Examples

```
old <- cl_options(verbose = FALSE)
cl_options()$verbose
cl_options(old)
```

`cl_persistence` *Estimate the temporal persistence of cloud from an observation record*

Description

Cloud occurrence is autocorrelated in time, and that autocorrelation degrades time-series retrievals largely independently of how much cloud there is: it concentrates observation loss into contiguous runs that can remove an entire seasonal transition while leaving the annual observation count almost unchanged. Quantifying it therefore requires its own estimator.

For the two-state chain used throughout cloudscape, with marginal cloud probability \{p\} and lag-1 autocorrelation \eqn\{\\rho\}, \deqn\{p_\{11\} = p + (1-p)\\rho, \\qquad p_\{01\} = p(1-\\rho),\} so that \eqn\{\\rho = p_\{11\} - p_\{01\}\}. The estimator is therefore the difference between the empirical probability that a cloudy acquisition is followed by another cloudy one and the probability that a clear acquisition is followed by a cloudy one. This requires no model fitting and is unbiased for a stationary chain.

Acquisitions are irregularly spaced, so transitions are counted only between consecutive acquisitions of the same sensor and only when the interval falls within \code{max_interval

Usage

```
cl_persistence(obs, threshold = 0.2, max_interval = 20, min_pairs = 10, by =
  c("all", "year", "season"))
```

Arguments

`obs`	A `cl_obs` table.
`threshold`	Cloud fraction above which an acquisition counts as cloudy.
`max_interval`	Longest gap in days across which a transition is still counted. Pairs separated by more than this are dropped, because a transition across a three-month gap carries no information about persistence at the acquisition scale.
`min_pairs`	Minimum usable transitions required to report an estimate.
`by`	Period granularity, or `"all"` to pool the whole record.

Value

A data frame with one row per cell, sensor and period, giving `p_cloud`, `p11`, `p01`, `rho`, `median_interval_days`, `n_pairs` and `mean_cloudy_run`, the mean length of consecutive cloudy acquisitions.

Attenuation

The estimate is a **lower bound** on the persistence that matters for a specific location. Whether an acquisition is usable at a given cell is inferred by thresholding a scene-level cloud fraction, which is a noisy proxy for the underlying state. Independent misclassification attenuates an estimated autocorrelation towards zero by approximately \{(1 - \\alpha - \\beta)^2\}, where \eqn\{\\alpha\} and \eqn\{\\beta\} are the two misclassification rates. In simulation with a realistic overlap between the cloudy and clear cloud-fraction distributions, a true \eqn\{\\rho\} of 0.6 is recovered as approximately 0.41 and a true 0.3 as approximately 0.20. Estimates from tier \code{"qa"

Examples

```
d <- seq(as.Date("2023-01-01"), as.Date("2023-12-31"), by = "5 days")
ts <- cl_simulate_series(d, p_cloud = 0.5, persistence = 0.6, seed = 1)
o <- cl_obs(1, ts$date, ts$cloud_fraction, sensor = "sentinel-2-msi")
cl_persistence(o)$rho
```

---

cl_pheno_curve *Double-logistic seasonal trajectory*

Description

The standard four-phase form used across land-surface phenology.

Usage

```
cl_pheno_curve(doy, base = 0.15, amplitude = 0.6, sos = 120, eos = 280,
  slope_up = 0.12, slope_down = 0.10)
```

Arguments

`doy`	Day of year.
`base`	Winter baseline value.
`amplitude`	Seasonal amplitude.
`sos`	Start and end of season, in days.
`eos`	Start and end of season, in days.
`slope_up`	Transition rates.
`slope_down`	Transition rates.

Value

Numeric vector of index values.

Examples

```
plot_ready <- cl_pheno_curve(1:365)
round(range(plot_ready), 3)
```

cl_pheno_fit *Fit a seasonal trajectory and extract phenological metrics*

Description

Fit a seasonal trajectory and extract phenological metrics

Usage

```
cl_pheno_fit(doy, value, model = c("dbl_logistic", "spline"), threshold =
  0.5, start = NULL)
```

Arguments

doy	Day of year of each observation.
value	Observed index values.
model	"dbl_logistic" or "spline".
threshold	Amplitude fraction defining SOS and EOS for the threshold extraction, following common practice.
start	Optional named list of starting values.

Value

A list with sos, eos, los, peak_doy, peak_value, converged and fit.

cl_pheno_map *Map phenological retrieval error across grid cells*

Description

Applies cl_pheno_power cell by cell using the actual acquisition dates and cloud fractions recorded for each cell, turning an observation table into a map of expected phenological uncertainty.

Usage

```
cl_pheno_map(obs, year = NULL, n_sim = 50L, ...)
```

Arguments

obs	A cl_obs table.
year	Year to analyse; defaults to the most frequent year present.
n_sim	Replicates per cell.
...	Passed to cl_pheno_power.

Value

A cl_stats table with metric sos_error_days, carrying a details attribute with the full per-cell summaries.

---

cl_pheno_power *Phenological retrieval error under realistic cloud loss*

Description

Simulates the effect of missing observations on phenological metric retrieval. For each replicate, a reference curve is sampled at the supplied acquisition dates, observations are dropped according to their cloud fraction, noise is added, the curve is refitted, and the retrieval error is recorded.

The output is directly interpretable: an expected absolute error in days for start of season, end of season and length of season, together with the failure rate, meaning the fraction of replicates in which the fit did not converge at all. In heavily clouded regions the failure rate is often the more important number, because a study there does not produce a biased estimate, it produces no estimate.

Usage

```
cl_pheno_power(dates, cloud_fraction = 0.4, threshold = 0.2, curve = NULL,
  noise = 0.03, n_sim = 100L, model = c("dbl_logistic", "spline"),
  persistence = 0.35, seed = NULL)
```

Arguments

dates	Acquisition dates available in the year.
cloud_fraction	Cloud fraction for each acquisition, or a single value.
threshold	Maximum cloud fraction for a usable observation.
curve	Function of day-of-year giving the reference trajectory, or a list of arguments passed to cl_pheno_curve.
noise	Observation noise standard deviation.
n_sim	Number of replicates.
model	Fitting model passed to cl_pheno_fit.
persistence	Lag-1 autocorrelation of cloud loss, in \[0, 1), using the same chain as cl_simulate_series. 0 gives independent loss, which is unrealistic and optimistic: independent loss thins the series uniformly and almost never removes a whole transition, whereas clustered loss regularly removes green-up entirely.
seed	Random seed.

Value

A list with summary (a one-row data frame of median absolute errors, interquartile ranges and failure rate) and replicates.

Examples

```
d <- seq(as.Date("2023-01-01"), as.Date("2023-12-31"), by = "10 days")
r <- cl_pheno_power(d, cloud_fraction = 0.5, n_sim = 30, seed = 1)
r$summary[, c("n_usable_median", "sos_mae", "failure_rate")]
```

`cl_probability` *Compute cloud probability*

Description

Compute cloud probability

Usage

```
cl_probability(x, method = "threshold", sensor = NULL, geom = NULL,
  threshold = 0.5, buffer = 0, ...)
```

Arguments

`x`	A `cl_scene`, or a named list of reflectance matrices.
`method`	Method name; see `cl_methods`.
`sensor`	Sensor id, required when `x` is a plain list.
`geom`	Optional list with `sun_zenith`, `sun_azimuth`, `view_zenith`, `view_azimuth`.
`threshold`	Probability threshold used to derive the class layer.
`buffer`	Dilate the resulting cloud mask by this many pixels. Cloud masks systematically under-detect at edges, so a small buffer is standard practice; the default of 0 leaves that decision explicit.
`...`	Passed to the method.

Value

A `cl_maskset`.

Examples

```
s <- cl_simulate(64, 64, coverage = 0.3, seed = 1,
                 background = list(blue = matrix(0.08, 64, 64),
                                   green = matrix(0.09, 64, 64),
                                   red = matrix(0.07, 64, 64),
                                   nir = matrix(0.35, 64, 64),
                                   swir16 = matrix(0.22, 64, 64)))
m <- cl_probability(s$bands, method = "threshold", sensor = "sentinel-2-msi")
round(mean(m$class), 2)
```

cl_project — *Project longitude/latitude to and from EASE-Grid 2.0 metres*

Description

Closed-form cylindrical equal-area projection on the WGS84 ellipsoid. These are provided directly rather than through PROJ so that the analysis grid, and the tests that verify it, work without a geospatial toolchain.

Usage

```
cl_project(lon, lat, lat_ts = 30)
cl_unproject(x, y, lat_ts = 30)
```

Arguments

`lon`	Longitude and latitude in degrees.
`lat`	Longitude and latitude in degrees.
`x`	Projected coordinates in metres.
`y`	Projected coordinates in metres.
`lat_ts`	Standard parallel in degrees. 30 gives EPSG:6933.

Value

A two-column matrix with columns `x`/`y` or `lon`/`lat`.

Examples

```
cl_project(c(0, 90), c(0, 45))
cl_unproject(cl_project(10, 50)[, 1], cl_project(10, 50)[, 2])
```

cl_qa_decode — *Decode a producer quality layer*

Description

Translates a bit-packed or categorical QA band into the standard class vocabulary of `cl_classes`, using the sensor driver's specification.

Usage

```
cl_qa_decode(qa, sensor, include_cirrus = TRUE, confidence = "medium")
```

Arguments

qa	QA matrix or SpatRaster.
sensor	Sensor id or driver.
include_cirrus	Treat flagged cirrus as cloud.
confidence	Minimum confidence level ("low", "medium", "high") for bit-packed QA with confidence fields.

Value

A list of logical matrices: cloud, shadow, snow, water, fill, plus class in the standard vocabulary.

---

cl_qa_spec *Describe a quality-assessment band*

Description

QA layers come in two flavours: bit-packed (Landsat QA_PIXEL) and categorical (Sentinel-2 SCL). cl_qa_spec() describes either so that cl_probability with method = "qa" can decode any sensor uniformly.

Usage

```
cl_qa_spec(type, asset, bits = NULL, confidence = NULL, classes = NULL,
  clear = NULL)
```

Arguments

type	"bitmask" or "categorical".
asset	Asset name of the QA layer.
bits	Named integer vector of bit positions (0-indexed) for type = "bitmask". Names should include, where available, cloud, cloud_shadow, snow, cirrus, water, fill, dilated_cloud.
confidence	Named list of two-bit confidence field positions, e.g. list(cloud = c(8, 9)), giving low/medium/high gradations.
classes	Named integer vector mapping class labels to codes for type = "categorical".
clear	Character vector of class names considered clear.

Value

An object of class cl_qa_spec.

cl_reliability *Flag conditions where cloud masks are known to be unreliable*

Description

Returns a per-pixel reliability score. Low values mark surfaces on which every published cloud mask has documented weaknesses, so that a downstream statistic can be reported with an honest caveat rather than an unqualified number.

Usage

```
cl_reliability(bands, dem = NULL, sun_zenith = NULL, sun_azimuth = NULL, res
  = 30)
```

Arguments

bands	Named list of reflectance matrices.
dem	Optional elevation matrix, used to flag terrain shadow risk.
sun_zenith	Illumination geometry, for terrain shadow.
sun_azimuth	Illumination geometry, for terrain shadow.
res	Pixel size in metres.

Value

A list with `reliability` in \[0, 1\] and named logical `flags`.

cl_require *Require a suggested package*

Description

cloudscape keeps its hard dependency surface small: the statistical core (availability, gaps, seasonality, phenology, evaluation, simulation) runs on base R alone, while raster and network functionality is gated behind suggested packages. This helper produces a single actionable error rather than an opaque failure deep in a call stack.

Usage

```
cl_require(pkg, reason = NULL)
```

Arguments

pkg	Package name(s).
reason	Short description of what the package is needed for.

Value

`TRUE` invisibly if all packages are available; otherwise an error.

`cl_roc` *ROC curve and area under it*

Description

ROC curve and area under it

Usage

```
cl_roc(prob, truth, n_thresh = 101)
```

Arguments

`prob`	Predicted probabilities in \[0, 1\].
`truth`	Binary reference (1 = positive).
`n_thresh`	Number of thresholds to evaluate.

Value

A data frame of `threshold`, `fpr`, `tpr`, `precision`, with an `auc` attribute.

Examples

```
set.seed(2)
truth <- rbinom(500, 1, 0.4)
prob  <- pmin(pmax(truth * 0.6 + runif(500) * 0.5, 0), 1)
attr(cl_roc(prob, truth), "auc")
```

`cl_scale` *Rescale stored integers to reflectance*

Description

Rescale stored integers to reflectance

Usage

```
cl_scale(x, sensor = NULL, clamp = c(0, 1.6))
```

Arguments

x	A cl_scene or named list of matrices.
sensor	Sensor id, when x is a list.
clamp	Clip to a physically plausible range.

Value

The rescaled object.

---

cl_scene *Construct a scene*

Description

A cl_scene bundles everything an algorithm needs for one acquisition. Reflectance is held as a terra::SpatRaster, which is itself lazy: pixels stay on disk until a computation forces them into memory. Attaching a DEM, solar geometry or QA layer never triggers a read.

Usage

```
cl_scene(reflectance = NULL, sensor, datetime = NULL, solar_geom = NULL,
  view_geom = NULL, dem = NULL, qa = NULL, masks = list(), footprint = NULL,
  id = NULL, manifest = NULL, ...)
```

Arguments

reflectance	A SpatRaster with layers named using standardised band names, or NULL.
sensor	Sensor id or cl_sensor driver.
datetime	Acquisition time (POSIXct or something coercible).
solar_geom	List with zenith and azimuth in degrees; either scalars (scene-mean) or SpatRaster layers (per-pixel).
view_geom	List with zenith and azimuth in degrees.
dem	Optional elevation SpatRaster aligned to reflectance.
qa	Optional QA SpatRaster.
masks	Named list of cl_maskset objects.
footprint	Optional lon/lat vertex matrix or sf geometry.
id	Scene identifier.
manifest	A cl_manifest.
...	Additional metadata stored verbatim.

Value

An object of class cl_scene.

cl_search *Search a catalogue for scenes*

Description

Queries a STAC API and returns a normalised item table. Provider-specific property names are translated to a common vocabulary so that downstream code never branches on the backend.

Usage

```
cl_search(aoi, sensor, start, end, max_cloud = 100, limit = 500, backend =
  NULL, extra = list())
```

Arguments

aoi	Area of interest: a numeric bounding box c(xmin, ymin, xmax, ymax) in degrees, or an sf object.
sensor	Sensor id or alias.
start	Date range.
end	Date range.
max_cloud	Maximum scene cloud cover in percent.
limit	Maximum items to return; Inf fetches all pages.
backend	Catalogue backend.
extra	Additional STAC query fields.

Value

A data frame of class cl_items with columns id, datetime, sensor, platform, cloud_cover, path_row or tile, geometry_bbox and assets.

cl_seasonality *Seasonal clear-sky model*

Description

Fits a harmonic logistic model of clear-sky probability against day of year for each cell, and reports the phase of the clear-sky maximum. The practical output is a best-acquisition-window map: the contiguous stretch of the year during which a cell is most likely to be observable.

Usage

```
cl_seasonality(obs, threshold = 0.2, harmonics = 1L, window_days = 60,
  min_obs = 12)
```

Arguments

obs	A cl_obs table.
threshold	Maximum cloud fraction for a usable observation.
harmonics	Number of annual harmonics (1 or 2).
window_days	Length of the reported best window.
min_obs	Minimum observations per cell required to attempt a fit.

Value

A data frame with one row per cell: amplitude, peak_doy, p_clear_peak, p_clear_trough, window_start, window_end, n, and converged.

---

cl_sensor_driver *Construct a sensor driver*

Description

A sensor driver is the single point of sensor-specific knowledge in cloudscape. Algorithms address bands by standardised names (blue, nir, swir16, cirrus, tir1, ...) and the driver translates those into the asset names a particular catalogue uses. Everything downstream is therefore sensor-agnostic.

Usage

```
cl_sensor_driver(id, name, bands, resolution = NULL, platforms = NULL,
  footprint = "none", revisit = NA_real_, qa = NULL, scale = 1, offset = 0,
  collections = list(), cloud_property = "eo:cloud_cover", has_thermal =
  FALSE, has_cirrus = FALSE, notes = NULL)
```

Arguments

id	Short identifier, e.g. "landsat-8-9-oli".
name	Human-readable name.
bands	Named character vector mapping standardised band names to the asset or layer names used by this sensor.
resolution	Named numeric vector of native resolutions in metres, using the same standardised band names.
platforms	data.frame with columns platform, start, end giving the operational period of each satellite in the constellation. end may be NA for currently active platforms. Used by cl_synergy to compute time-varying revisit.
footprint	Footprint geometry type: "wrs2", "mgrs", "sinusoidal", "scene" or "none".
revisit	Nominal revisit interval in days for a single platform.

qa	Optional QA specification created by cl_qa_spec.
scale	Linear rescaling from stored integers to reflectance.
offset	Linear rescaling from stored integers to reflectance.
collections	Named list mapping catalogue backends to collection ids.
cloud_property	Name of the scene-level cloud-cover property in catalogue metadata.
has_thermal	Logical capability flags. Algorithms that require these bands check the flags and degrade gracefully.
has_cirrus	Logical capability flags. Algorithms that require these bands check the flags and degrade gracefully.
notes	Free text recorded in manifests.

Value

An object of class cl_sensor.

Examples

```
drv <- cl_sensor("landsat-8-9-oli")
drv$bands[["cirrus"]]
```

---

cl_shadow *Detect cloud shadows*

Description

High-level entry point combining a darkness index, geometric projection and optional thermal height constraint.

Usage

```
cl_shadow(cloud, nir = NULL, swir = NULL, bt = NULL, water = NULL, res = 30,
  sun_zenith = 40, sun_azimuth = 180, view_zenith = 0, view_azimuth = 0,
  method = c("geometric", "darkness", "union"), heights = seq(200, 12000, by
  = 200), buffer = 0, dark_threshold = 0.12)
```

Arguments

cloud	Logical/0-1 cloud matrix or SpatRaster.
nir	Reflectance matrices used to build the darkness index. NIR alone is workable; adding SWIR sharply reduces confusion with water.
swir	Reflectance matrices used to build the darkness index. NIR alone is workable; adding SWIR sharply reduces confusion with water.
bt	Optional brightness temperature for height estimation.
water	Optional logical matrix of water pixels, excluded from shadow.

res	Pixel size in metres.
sun_zenith	Geometry in degrees.
sun_azimuth	Geometry in degrees.
view_zenith	Geometry in degrees.
view_azimuth	Geometry in degrees.
method	"geometric" searches heights against a darkness layer; "darkness" thresholds the darkness index alone, ignoring geometry; "union" returns the geometric union across all heights.
heights	Candidate heights in metres.
buffer	Dilate the resulting shadow by this many pixels.
dark_threshold	Relative darkening counted as shadow; see cl_shadow_project.

Value

A cl_maskset with the shadow layer populated.

---

cl_shadow_offset *Ground displacement of a cloud shadow*

Description

For a cloud top at height height above the surface, illuminated with solar zenith angle sun_zenith and solar azimuth sun_azimuth, the shadow falls at a horizontal distance height * tan(sun_zenith) in the direction the sunlight travels, that is, sun_azimuth + 180 degrees.

A second, smaller displacement arises because the satellite does not view the cloud from nadir: an elevated cloud is imaged away from the ground point it actually sits above. Landsat's field of view is narrow enough that this is often neglected, but across a Sentinel-2 swath it reaches roughly 0.19 times the cloud height and is worth correcting.

Usage

```
cl_shadow_offset(height, sun_zenith, sun_azimuth, view_zenith = 0,
  view_azimuth = 0, correct_parallax = TRUE)
```

Arguments

height	Cloud-top height above the surface, in metres.
sun_zenith	Solar zenith angle in degrees (0 = overhead).
sun_azimuth	Solar azimuth in degrees clockwise from north, giving the compass direction *of* the sun.
view_zenith	Sensor viewing geometry in degrees. Defaults assume nadir viewing.
view_azimuth	Sensor viewing geometry in degrees. Defaults assume nadir viewing.
correct_parallax	Apply the viewing-geometry correction.

Value

A matrix with columns `dx` (metres east) and `dy` (metres north), giving the offset from the observed cloud position to the shadow position.

Examples

```
# Sun due south at 45 degrees zenith: a 1 km cloud casts a 1 km shadow north
cl_shadow_offset(1000, sun_zenith = 45, sun_azimuth = 180)

# Sun in the east: shadow falls to the west
cl_shadow_offset(1000, sun_zenith = 45, sun_azimuth = 90)
```

`cl_shadow_project` *Project a cloud mask to candidate shadow locations*

Description

Sweeps a range of plausible cloud-top heights, displaces the cloud mask by the corresponding ground offset at each height, and returns either the height that best matches a supplied darkness layer or the union of all candidate positions.

The similarity criterion follows the standard formulation: for each height, score the fraction of projected pixels that are actually dark, and take the height maximising that fraction. When no darkness layer is supplied the function returns a conservative union mask, which is appropriate for screening but will over-mask.

Usage

```
cl_shadow_project(cloud, darkness = NULL, res = 30, sun_zenith = 40,
  sun_azimuth = 180, view_zenith = 0, view_azimuth = 0, heights = seq(200,
  12000, by = 200), dark_threshold = 0.12)
```

Arguments

`cloud`	Logical or 0/1 matrix of cloud pixels.
`darkness`	Optional numeric matrix in \[0, 1\] where high values indicate potential shadow (dark in NIR/SWIR). Required for height search.
`res`	Pixel size in metres.
`sun_zenith`	Illumination geometry in degrees.
`sun_azimuth`	Illumination geometry in degrees.
`view_zenith`	Viewing geometry in degrees.
`view_azimuth`	Viewing geometry in degrees.
`heights`	Candidate cloud-top heights in metres.
`dark_threshold`	Relative darkening above which a pixel counts as shadowed when scoring a candidate height. 0.12 means the pixel is at least 12 percent darker than the clear-sky reference.

Value

A list with `shadow` (0/1 matrix), `height` (best height in metres, `NA` when no darkness layer was supplied), `score` (matching fraction), `retained` (fraction of the projected shadow still inside the window at the selected height), `truncated` (logical; `TRUE` when no candidate height keeps at least half its shadow inside the window, in which case the height is not identifiable), `window_m`, `required_m`, `resolvable_m` (the largest cloud-top height this window can resolve), and `profile` (a data frame of score and retention against height).

Examples

```
cloud <- matrix(0, 40, 40); cloud[18:22, 18:22] <- 1
dark  <- matrix(0, 40, 40); dark[8:12, 18:22] <- 1   # shadow 10 cells north
r <- cl_shadow_project(cloud, dark, res = 100, sun_zenith = 45,
                       sun_azimuth = 180, heights = seq(200, 2000, 100))
r$height
```

`cl_simulate` *Simulate clouds and cloud shadows*

Description

Generates a synthetic cloud field with controllable coverage, size, edge softness and opacity, casts a geometrically consistent shadow, and optionally composites both onto a background reflectance scene.

Opacity is the key difficulty control. Fully opaque cloud is easy for every method; the discrimination between algorithms happens between roughly 0.2 and 0.6 opacity, which is where thin cirrus and cloud edges live and where published masks disagree most.

Usage

```
cl_simulate(nrow = 256, ncol = 256, background = NULL, coverage = 0.25, size
  = 0.15, opacity = c(0.3, 1), edge_softness = 3, height = 2000, res = 30,
  sun_zenith = 40, sun_azimuth = 150, view_zenith = 0, view_azimuth = 0,
  shadow_darkening = 0.45, cloud_reflectance = 0.8, noise = 0.005, seed =
  NULL)
```

Arguments

`nrow`	Scene dimensions in pixels. Ignored if `background` is given.
`ncol`	Scene dimensions in pixels. Ignored if `background` is given.
`background`	Optional named list or 3-D array of clear reflectance bands to composite onto. Band names should be standardised.
`coverage`	Target cloud fraction in \[0, 1\].
`size`	Characteristic cloud size as a fraction of scene width; smaller values give more, smaller clouds.

opacity	Cloud opacity in \[0, 1\]; 1 is fully opaque. May be a length-2 vector giving a range, sampled per cloud object.
edge_softness	Width of the semi-transparent cloud edge, in pixels.
height	Cloud-top height in metres.
res	Pixel size in metres.
sun_zenith	Geometry in degrees.
sun_azimuth	Geometry in degrees.
view_zenith	Geometry in degrees.
view_azimuth	Geometry in degrees.
shadow_darkening	Multiplicative reflectance factor inside shadow.
cloud_reflectance	Reflectance of fully opaque cloud.
noise	Standard deviation of additive Gaussian sensor noise.
seed	Random seed.

Value

A list with `cloud` (0/1 truth), `opacity` (continuous), `shadow` (0/1 truth), `truth` (class codes from `cl_classes`), `bands` (list of contaminated reflectance matrices, when `background` was supplied) and `params`.

Examples

```
s <- cl_simulate(120, 120, coverage = 0.3, opacity = c(0.4, 1), seed = 42)
round(mean(s$cloud), 2)
sum(s$shadow) > 0
```

---

cl_simulate_series *Simulate a contaminated time series*

Description

Cloud occurrence is temporally autocorrelated: a cloudy day is followed by a cloudy day more often than chance. Benchmarks built on independent draws therefore flatter any gap-filling or compositing method, because independent cloud almost never produces the long runs that break real time series. This function generates cloud occurrence from a two-state Markov chain whose persistence is set explicitly.

Usage

```
cl_simulate_series(dates, p_cloud = 0.5, persistence = 0.35, seed = NULL)
```

Arguments

dates	Acquisition dates.
p_cloud	Marginal probability of a cloudy acquisition.
persistence	Lag-1 autocorrelation of the cloudy/clear sequence, in \[0, 1). 0 gives independent draws; 0.35 is a reasonable default for acquisitions spaced a few days apart. Parameterising by correlation rather than by a transition multiplier keeps the marginal probability exact at every value, which a multiplier does not: multipliers must be clamped at high cloud fractions, and the clamped chain then mixes so slowly that a one-year series never leaves its initial state.
seed	Random seed.

Value

A data frame with `date`, `cloudy` and `cloud_fraction`.

Examples

```
d <- seq(as.Date("2023-01-01"), as.Date("2023-12-31"), by = "5 days")
ts <- cl_simulate_series(d, p_cloud = 0.6, persistence = 1.4, seed = 1)
mean(ts$cloudy)
```

`cl_solar_position` *Solar position for a location and time*

Description

A compact solar position algorithm, accurate to a few hundredths of a degree over the satellite era, used when a scene lacks per-pixel angle grids.

Usage

```
cl_solar_position(datetime, lon, lat)
```

Arguments

datetime	`POSIXct` in UTC.
lon	Coordinates in degrees.
lat	Coordinates in degrees.

Value

A data frame with `zenith` and `azimuth` in degrees.

Examples

```
cl_solar_position(as.POSIXct("2023-06-21 12:00:00", tz = "UTC"), 0, 51.5)
```

`cl_stats` *Construct a grid statistics table*

Description

`cl_stats` is a long-format table: one row per cell, period, sensor and metric. Every row must declare its `unit`, its sample size `n`, and the `denominator` the value is expressed per. The constructor validates these against a controlled vocabulary and refuses out-of-range values.

This strictness is deliberate. A count of images summed over ten years and then divided by ten is a mean *per year*, not a mean *per scene footprint*; presenting one as the other produces values that are physically impossible for the stated denominator. Encoding the denominator in the data structure makes that class of error a validation failure rather than a published table.

Usage

```
cl_stats(cell, metric, value, period = NA_character_, sensor =
  NA_character_, n = NA_integer_, se = NA_real_, denominator = "cell-year",
  grid = NULL, manifest = NULL)
```

Arguments

`cell`	Integer grid cell ids.
`metric`	Metric name; see `cl_metrics`.
`value`	Numeric values.
`period`	Character period labels, e.g. `"2023"` or `"2023-06"`.
`sensor`	Sensor ids.
`n`	Sample size behind each value.
`se`	Optional standard errors.
`denominator`	What the value is expressed per: one of `"cell"`, `"cell-year"`, `"cell-month"`, `"scene"`, `"pixel"`.
`grid`	Optional `cl_grid` the cells refer to.
`manifest`	A `cl_manifest`.

Value

An object of class `cl_stats`, which inherits from `data.frame`.

Examples

```
s <- cl_stats(cell = 1:3, metric = "cloud_fraction", value = c(.2, .5, .9),
              period = "2023", sensor = "sentinel-2-msi", n = c(40, 38, 41),
              denominator = "cell-year")
summary(s)
```

cl_stats_merge *Combine statistics tables*

Description

Merges rows describing the same cell, period, sensor and metric, using the combination rule declared for each metric in cl_metrics: counts add, fractions and rates combine as sample-size-weighted means, and maximum gap takes the maximum.

This is for combining *disjoint* evidence, such as separate years, sensors or spatial partitions. It cannot repair double counting: if the same acquisition was aggregated into two inputs, summing the counts will inflate them. Deduplicate at the observation level before aggregating.

Usage

```
cl_stats_merge(..., grid = NULL)
```

Arguments

...	cl_stats tables.
grid	Optional cl_grid recorded on the result.

Value

A single cl_stats table.

Examples

```
a <- cl_stats(1:2, "n_scenes", c(10, 12), period = "2023", n = c(10, 12))
b <- cl_stats(1:2, "n_scenes", c(5, 6), period = "2023", n = c(5, 6))
cl_stats_merge(a, b)$value
```

cl_stats_wide *Reshape statistics from long to wide*

Description

Reshape statistics from long to wide

Usage

```
cl_stats_wide(x, metrics = NULL)
```

Arguments

x	A cl_stats.
metrics	Metrics to include; defaults to all.

Value

A data frame with one row per cell/period/sensor and one column per metric.

`cl_synergy` *Multi-constellation revisit synergy*

Description

Answers the question a study designer actually asks: if I combine these sensors, how many observations do I get, and how does that change through time?

Effective revisit is not a constant. Landsat 9 joined Landsat 8 in late 2021, roughly halving the Landsat interval from that point; Sentinel-2B joined 2A in 2017 and 2C in 2024. A study spanning 2014 to the present therefore has a non-stationary sampling density, and comparing early and late years without accounting for it confounds acquisition capacity with whatever is being measured.

Usage

```
cl_synergy(sensors, start, end, by = c("year", "month"), latitude = 0)
```

Arguments

`sensors`	Character vector of sensor ids or aliases.
`start`	Analysis period.
`end`	Analysis period.
`by`	Reporting granularity, `"year"` or `"month"`.
`latitude`	Optional latitude used to inflate revisit for orbital side-lap, which increases towards the poles.

Value

A data frame with columns `period`, `sensor`, `n_platforms`, `nominal_obs`, `combined_obs` and `effective_revisit_days`.

Examples

```
cl_synergy(c("landsat-8-9-oli", "sentinel-2-msi"),
           start = "2015-01-01", end = "2025-12-31", by = "year")[1:6, ]
```

cl_terrain_shadow *Cast shadow from terrain*

Description

Terrain shadow is routinely confused with cloud shadow in rugged country: both are dark, both correlate with slope aspect, and neither Fmask, Sen2Cor's scene classification nor single-date neural masks separate them explicitly. Computing terrain shadow from a DEM lets `cl_shadow` exclude it, and lets `cl_reliability` flag where the distinction is hard.

Implemented by marching along the solar ray from each pixel and testing whether any upstream terrain rises above the ray.

Usage

```
cl_terrain_shadow(dem, sun_zenith, sun_azimuth, res = 30, max_distance =
  20000)
```

Arguments

`dem`	Elevation matrix in metres.
`sun_zenith`	Illumination geometry in degrees.
`sun_azimuth`	Illumination geometry in degrees.
`res`	Pixel size in metres.
`max_distance`	Maximum ray length in metres.

Value

A 0/1 matrix, 1 where terrain blocks direct illumination.

Examples

```
dem <- matrix(0, 30, 30); dem[, 20:30] <- 500   # a ridge on the east side
ts <- cl_terrain_shadow(dem, sun_zenith = 70, sun_azimuth = 90, res = 30)
sum(ts) > 0
```

cl_validate *Accuracy metrics for a binary or multi-class mask*

Description

Accuracy metrics for a binary or multi-class mask

Usage

```
cl_validate(pred, truth, positive = 1, levels = NULL)
```

Arguments

pred	Predicted mask.
truth	Reference mask.
positive	For binary input, the value treated as the positive class.
levels	Optional class levels for multi-class input.

Value

A data frame of per-class precision, recall, F1 and IoU plus overall and balanced accuracy, with an attribute confusion.

Examples

```
set.seed(1)
truth <- matrix(rbinom(400, 1, 0.3), 20, 20)
pred  <- truth; pred[sample(400, 40)] <- 1 - pred[sample(400, 40)]
cl_validate(pred, truth)
```

---

cl_visualize *Quick look at a mask or scene*

Description

Quick look at a mask or scene

Usage

```
cl_visualize(x, what = c("probability", "class", "shadow"), ...)
```

Arguments

x	A cl_maskset, cl_scene or matrix.
what	For a maskset, "probability", "class" or "shadow".
...	Passed to [graphics::image()].

Value

Invisibly x.

---

length.cl_cube *length.cl_cube*

Usage

```
length.cl_cube(x)
```

print.cl_cube *print.cl_cube*

Usage

```
print.cl_cube(x, ...)
```

print.cl_grid *print.cl_grid*

Usage

```
print.cl_grid(x, ...)
```

print.cl_items *print.cl_items*

Usage

```
print.cl_items(x, n = 6L, ...)
```

print.cl_manifest *print.cl_manifest*

Usage

```
print.cl_manifest(x, ...)
```

print.cl_maskset *print.cl_maskset*

Usage

```
print.cl_maskset(x, ...)
```

print.cl_obs *print.cl_obs*

Usage

```
print.cl_obs(x, n = 6L, ...)
```

print.cl_scene *print.cl_scene*

Usage

```
print.cl_scene(x, ...)
```

print.cl_sensor *print.cl_sensor*

Usage

```
print.cl_sensor(x, ...)
```

print.cl_stats *print.cl_stats*

Usage

```
print.cl_stats(x, n = 6L, ...)
```

print.cl_validation *print.cl_validation*

Usage

```
print.cl_validation(x, ...)
```

sensor-registry *Register, retrieve and list sensor drivers*

Description

Register, retrieve and list sensor drivers

Usage

```
cl_sensor_register(driver, overwrite = FALSE)
cl_sensor(id)
cl_sensors()
```

Arguments

driver	A cl_sensor object.
id	Sensor identifier.
overwrite	Replace an existing registration.

Value

cl_sensor_register() returns the id invisibly; cl_sensor() returns a cl_sensor; cl_sensors() returns a data frame summary.

summary.cl_stats *summary.cl_stats*

Usage

```
summary.cl_stats(object, ...)
```

validate_cl_stats *Validate a statistics table*

Description

Validate a statistics table

Usage

```
validate_cl_stats(df)
```

Arguments

`df`	A data frame or `cl_stats`.

Value

`TRUE` invisibly, or an error describing the first violation.

Index